

MODULE 1

Introduction to **AI-Driven Low-Code Engineering**



2 HOURS

Ruthran Raghavan | Chief AI Scientist

ruthranraghavan.com

What We'll Cover Today

01

Evolution of Development

Traditional → Low-Code → AI-Powered

02

What is Vibe Coding?

Natural language app building

03

AI Agents

Role in application development

04

Core Concepts

Dataverse, MCP, RAG, Governance, ALM

05

Low-Code vs Pro-Code

Choosing the right approach

06

vibe.powerapps.com

AI-first platform, export & hands-on



The Evolution of Development

From Traditional to AI-Powered

Traditional Software Development

1990 – 2010

Characteristics

- Manual coding
- Long development cycles
- Specialized developers required
- High development costs

Technology Stack

Frontend: HTML, CSS, JavaScript

Backend: Java, .NET, PHP

Database: SQL Server, Oracle, MySQL

Development Timeline



Time Required: 3 – 12 Months

Low-Code Development

2010 - 2023

Characteristics

- Visual drag-and-drop interfaces
- Prebuilt connectors
- Citizen developers enabled
- Faster delivery

Popular Platforms

Microsoft Power Apps

OutSystems

Mendix

AppSheet

Development Timeline



Time Required: Days to Weeks

AI-Powered Development

2024 and Beyond

Characteristics

Natural language instructions

AI-generated applications

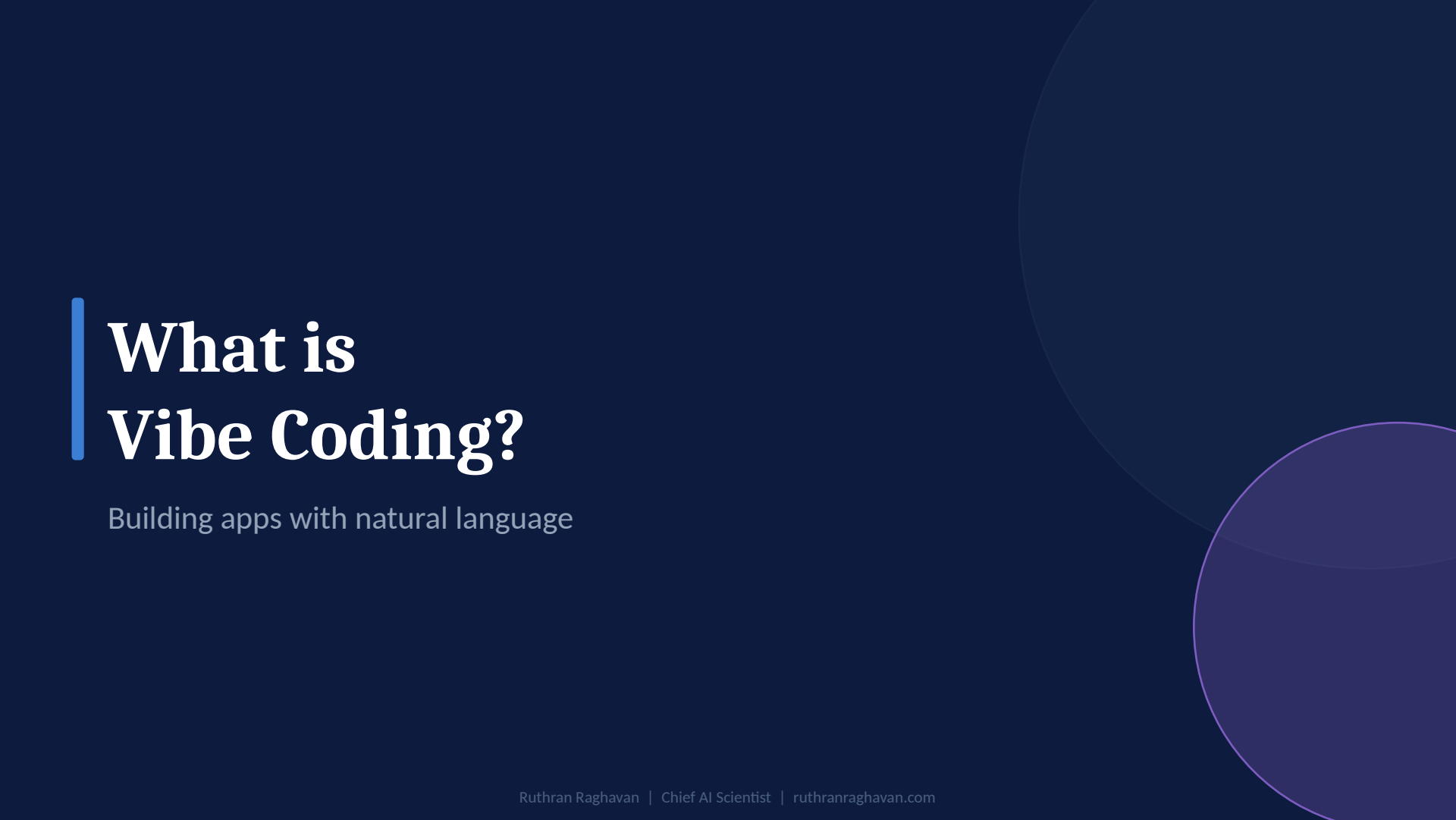
Autonomous agents

Intelligent automation

Development Timeline

- Business Requirement
- Natural Language Prompt
- AI Agent
- Generated Application
- Refinement
- Deployment

 Time Required: Minutes to Hours



What is Vibe Coding?

Building apps with natural language

Traditional Approach vs. Vibe Coding

Traditional

Developer: "I need a form."

Developer manually creates:

- Tables
- Forms
- Validation
- Workflows
- Security

— *Manually.*

VS

Vibe Coding

User Prompt:

"Create an HR onboarding app where candidates submit resumes, HR reviews, managers approve, and offer letters are auto-generated."

AI automatically generates:

- Database schema
- Forms
- Approval workflows
- Dashboards
- AI agents
- Security roles

— *Automatically.*

Natural Language as a Development Interface



Traditional Programming

```
if(employee.experience > 2) {  
  approveCandidate  
();  
}
```

Requires programming knowledge — specialized skill, time-consuming, error-prone.



Natural Language Development

Example Prompt:

"Automatically approve candidates with more than 2 years of experience and send an offer letter."

AI converts this into:

- Business logic
- Workflows
- Automations
- Data rules

No programming knowledge required — just describe what you want to build!

Effective Prompt Structure

1

Business Goal

State the overall objective

"Build an internship recruitment system."

2

Users

Define who will use the application

"Candidates, HR, Managers."

3

Data

List the key data entities needed

"Candidates, Applications, Offers."

4

Workflow

Describe the step-by-step process

"Apply → Review → Approve → Offer Letter."

5

Security

Specify access and permissions

"Candidates see only their own data."



Role of AI Agents in App Development

Intelligent · Autonomous · Dynamic

What is an AI Agent?

An AI Agent is a software system capable of understanding context, taking actions, making decisions, using tools, and completing tasks autonomously.



Understanding Context

Reads and interprets data, situations, and requirements from multiple sources



Taking Actions

Executes tasks, runs tools, and triggers automated workflows



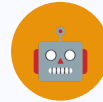
Making Decisions

Reasons through options and selects the best path based on goals



Using Tools

Connects to APIs, databases, and external systems seamlessly



Autonomous Completion

Works independently without step-by-step human guidance

Traditional Automation vs. AI Agent

Traditional Automation

Trigger: Application submitted

Action: Send Email

Fixed, linear workflow.

VS

AI Agent

Trigger: Application submitted

Agent Actions:

1. Analyze resume
2. Score candidate
3. Generate summary
4. Recommend approval
5. Draft offer letter
6. Notify stakeholders

Dynamic, intelligent workflow.

How AI Agents Work



OBSERVE

Reads Data

Processes inputs from databases, APIs, emails, and user actions to understand the current context.



REASON

Makes Decisions

Analyzes context, evaluates options, and determines the best course of action based on goals and constraints.



ACT

Uses Tools

Executes tasks: sends emails, creates records, triggers workflows, calls APIs, and generates documents.



LEARN

Improves Over Time

Adapts based on feedback and outcomes, continuously refining its decisions and performance.

Core Concepts Overview

Dataverse · MCP · RAG · Governance · ALM

Dataverse

Microsoft's enterprise data platform — "The SQL Database for Power Platform."

What Dataverse Stores

► Employees

► Students

► Customers

► Applications

► Transactions

Key Benefits

Security:

Row-level and field-level access control built in

Relationships:

Link tables with referential integrity

Audit Trail:

Track all data changes automatically

Scalability:

Enterprise-grade performance and capacity

MCP — Model Context Protocol

A standard way for AI agents to communicate with external systems and tools.



Think of MCP as "USB-C for AI Tools" — one universal connector for all systems.

HR System

ERP

CRM

MCP

Protocol

SharePoint

Power Apps

Dataverse

MCP provides a common language — one protocol to connect any AI agent to any system.

RAG — Retrieval Augmented Generation

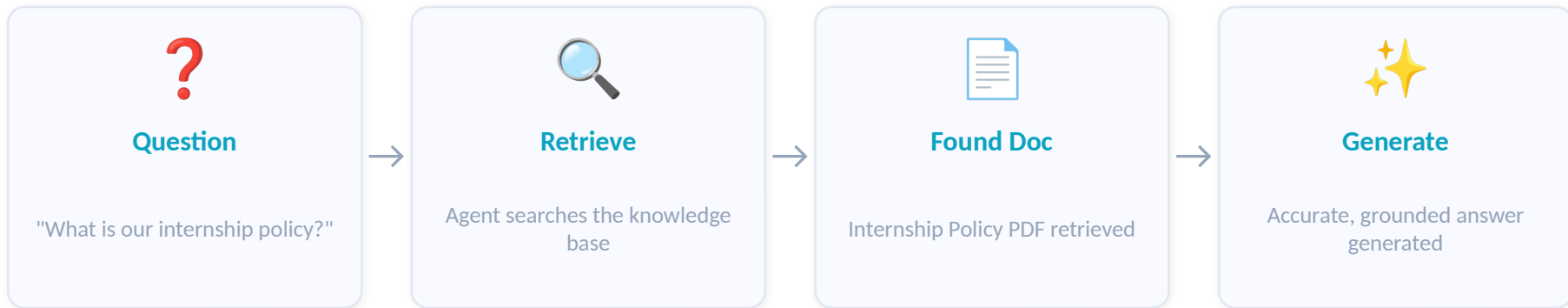
The Problem

LLMs do not know your company-specific data — policies, internal documents, SOPs, or proprietary knowledge bases.

The Solution

RAG allows agents to first retrieve relevant documents from your knowledge base, then generate accurate, grounded responses.

How RAG Works:



Governance

Purpose: Control and secure AI usage — ensuring data privacy, compliance, and proper access control.

Governance Covers



Data Privacy:

Protect sensitive information, ensure regulatory compliance



Access Control:

Define who can access which data and applications



Compliance:

Meet organizational and regulatory standards



Auditing:

Track all actions and changes for full accountability



Security:

Protect against unauthorized access and threats

Key Governance Questions

1. Who can access data?

2. Who can deploy applications?

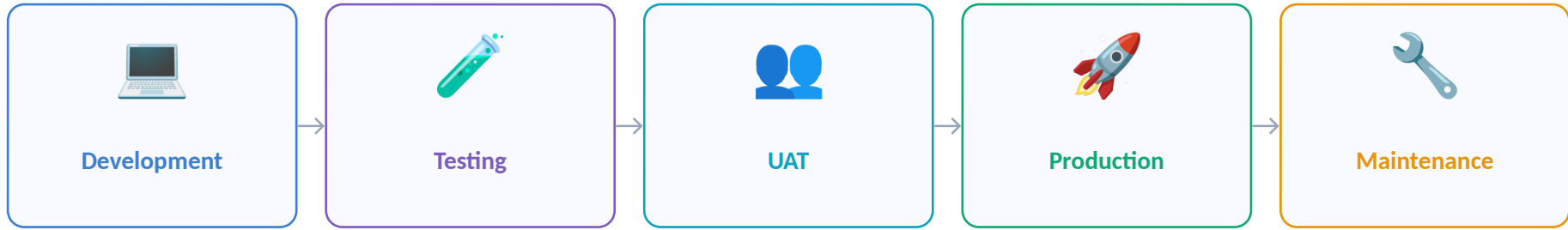
3. Who can create AI agents?

4. Who can use AI features?

ALM — Application Lifecycle Management

Managing applications throughout their entire lifecycle — from development to production and ongoing maintenance.

Lifecycle Stages



ALM Benefits

✓ **Version Control:**

Track all changes with complete history

✓ **Rollback:**

Quickly revert to any previous version

✓ **Deployment Automation:**

Consistent, repeatable deployments

✓ **Team Collaboration:**

Multiple developers work in parallel

Low-Code vs Pro-Code

Choosing the right tool for the job

When to Use Low-Code vs. Pro-Code

Use Low-Code When:

- Internal business applications
- HR systems
- Approval workflows
- CRM solutions
- Dashboards
- Data entry systems

Examples:

1. Leave Management
2. Employee Onboarding
3. Asset Tracking
4. Visitor Management

VS

Use Pro-Code (React + TypeScript) When:

- Consumer-facing applications
- Complex UI requirements
- High-performance systems
- Custom frameworks
- Large-scale SaaS products

Examples:

1. E-commerce platform
2. Social media application
3. Netflix-like application

The Hybrid Approach

Modern enterprises combine the best of both worlds:



Frontend:

React

Rich, complex UI for consumer-facing surfaces



Workflow:

Power Automate

Business process automation and approvals



Database:

Dataverse

Enterprise-grade secure data management



AI Layer:

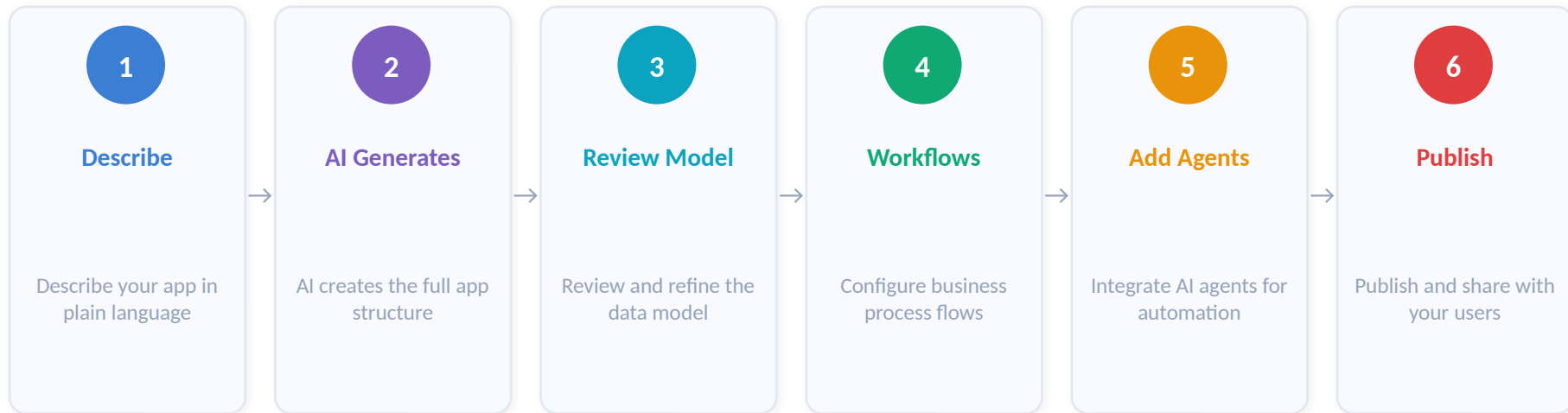
Copilot Agents

Intelligent automation and natural language

vibe.powerapps.com

An AI-first Power Apps development environment — create applications using natural language prompts.

Typical Workflow:



Export Capabilities

Generated solutions can be exported as:



Power Platform Solutions

Complete packaged solutions for deployment across Development, Test, and Production environments.



Dataverse Schema

Enterprise data model definitions for robust data management and governance across environments.



Power Automate Flows

Workflow automation definitions that can be versioned, shared, and reused across multiple projects.



Source Code Assets

Raw code files for advanced customization by pro-code developers using React or TypeScript.

Module 1: Hands-On Lab

Review a sample business requirement and identify:



App Features:

What screens, forms, and functions are needed?



Data Entities:

What tables and relationships are required?



Agent Actions:

What AI tasks should be automated?



External Tools:

What MCP integrations are needed?



Governance Concerns:

Who accesses what, and how is it secured?

Expected Outcome: Participants understand the end-to-end training journey and the final project expectations.